



Attention Is All You Need

Vaswani et al., 2017

0. Abstract

1. Introduction

2. Background

3. Model Architecture

4. Why Self-Attention

7. Conclusion

▼ structure

[기존 문제]

RNN / LSTM

↓

순차 계산 필요

↓

병렬화 어려움

↓

Long-range dependency도 어려움

[아이디어]

Attention은 멀리 떨어진 token도
직접 연결 가능하다.

"그럼 Attention만 쓰면?"

[제안]

Transformer

Embedding

+

Positional Encoding

↓

Multi-Head Self-Attention

↓

Feed Forward

↓

Encoder / Decoder

[장점]

- ① 병렬화
- ② 짧은 dependency path
- ③ 높은 번역 성능
- ④ 빠른 학습

[결과]

RNN / CNN 없이도

Attention-only architecture가 가능

0. Abstract

- 당시 주류의 sequence transduction model은 encoder, decoder를 포함한 RNN, CNN을 기반으로 함
 - 기계적 번역 모델이 대표적인 예시
- attention은 RNN encoder-decoder를 보조하는 역할 (input → encoder → attention → decoder → output)
- **Transformer**라는 모델을 제안: based **solely on attention**, w/o recurrence and convolutions

these models to be superior in quality while being more parallelizable and requiring significantly less time to train.

성능 + 병렬화 + 학습 효율 의 장점

1. Introduction

- 기존의 “RNN 발전시키기”연구가 주류인 상황에서 “RNN 자체가 필요한가?” 라는 의문을 제시함.
- RNN의 근본적인 문제인 병렬 연산 불가를 해결하고자 attention을 제시함



"The animal didn't cross the street because it was too tired."

Attention은 위 문장에서 **animal**과 **it**의 관계를 직접 계산할 수 있다. (RNN은 순차적으로 계산해야함)

- but, 기존 attention은 RNN과 함께 활용하는 구조 → 이를 해결하고자 Transformer를 제시한다.

2. Background

- 기존의 연구에서도 CNN기반으로 RNN의 순차 연산을 줄이고자 하는 시도가 많았음. But...

In these models(CNN base model), the number of operations required to relate signals from two arbitrary input or output positions grows in the distance between positions

→ 연산 횟수가 두 위치 사이의 거리가 멀어질수록 증가함. (animal ↔ it >> animal ↔ didn't)

기존의 $O(n)$, $O(\log n)$ 수준의 연산을 Transformer에서는 $O(1)$ 상수 수준으로 연산 가능.

- 다만 self-attention에서는 여러 value의 가중합을 이용하기에 서로 다른 종류의 관계가 섞여버릴 수 있다. → Multi-head attention을 활용하여 각 head가 서로 다른 관계를 보도록하며 제한적으로 활용하고 있었다.
- Transformer의 차별점으로, RNN 및 CNN에 의존하지 않고 self-attention만을 이용하는 최초의 transduction model임을 강조한다.

3. Model Architecture

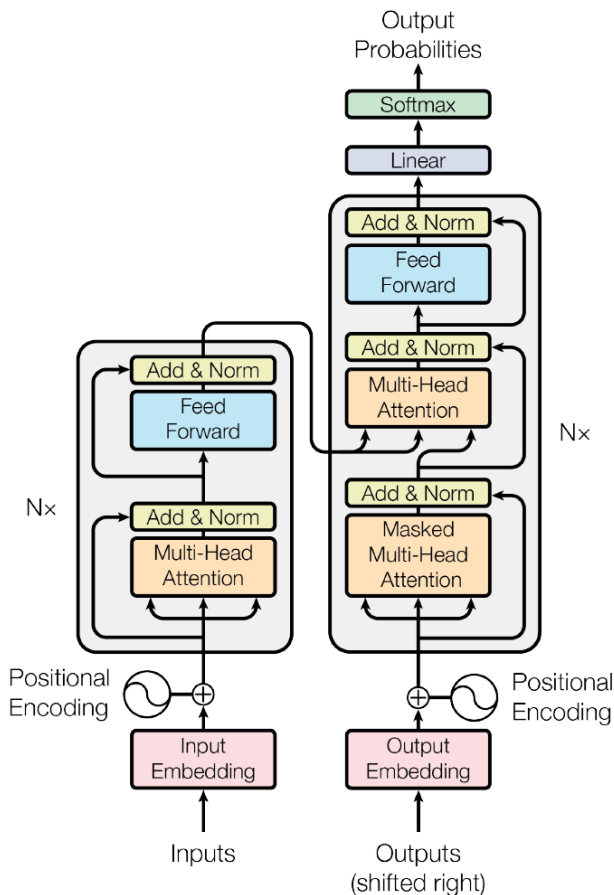


Figure 1: The Transformer - model architecture.

- Encoder (왼쪽 박스)

첫 번째 주황색 Sub-Layer는 **Multi-Head Encoder Self-Attention**, 파란색 Sub-Layer는 단순한 Feed-Forward입니다. 두 Sub-Layer 각각 Residual Connection을 사용하고 Layer Normalization을 사용합니다. N 개의 동일한 층이 쌓이고 마지막 Layer가 Decoder의 **Multi-Head Encoder-Decoder Attention**으로 입력됩니다. 본 논문에서는 $N=6$ 으로 동일한 6개의 층을 쌓아 올렸고 출력 차원은 512로 설정하였습니다. 처음 Encoder에 입력부터 출력까지 총 512차원으로 고정됩니다.

- Decoder (오른쪽 박스)

첫 번째 주황색 Sub-Layer는 **Masked Multi-Head Decoder Self-Attention**, 두 번째 주황색 Sub-Layer는 Encoder에서의 출력으로 입력된 **Multi-Head Encoder-Decoder Attention**, 파란색 Sub-Layer는 마찬가지로 단순한 Feed-Forward이고 Residual Connection과 Layer Normalization으로 동일하게 구성됩니다.

- Positional Encoding

Transformer는 RNN처럼 sequence 정보를 가지고 있지 않기 때문에, 문장에서 단어의 위치 정보를 추가해준다. → Attention의 단점을 보완

- Multi-head Attention

- Self-attention

문장의 각 token이 같은 문장 안의 다른 token들을 얼마나 참고해야 하는지 계산하는 과정



"The animal didn't cross the street because it was too tired."

에서 it → animal의 관계에 높은 attention을 부여할 수 있음

- Multi-head attention

it → animal같은 대명사-선행사 관계 외에 다양한 관계의 attention을 학습하기에 용이함.

- Masked Multi-head Attention (decoder)

decoder가 "나는 너를 사랑한다"를 예측해야 할 때, '너를'을 예측할 때 '사랑한다'를 보지 않도록 masking해줌.

- Free forward

attention을 거친 각 토큰의 representation을 추가로 변환(개별적인 가공)

- Attention

Query와 Key-Value pair를 받아 output을 반환하는 함수, output은 value들의 가중합으로 계산됨.

- **Query (Q)**: 내가 지금 찾고 싶은 정보
 - **Key (K)**: 각 정보가 무엇과 관련 있는지를 판단하기 위한 기준
 - **Value (V)**: 실제로 가져올 정보



"The animal didn't cross the street because it was too tired."

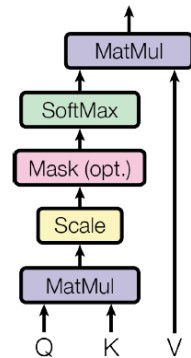
it이라는 token의 representation을 계산한다.

이때

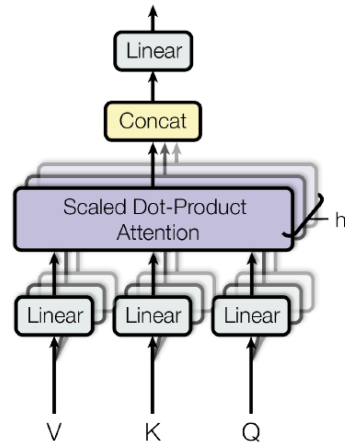
- **it**에서 나온 것이 **Query**
 - **The**, **animal**, **was**, **tired** ... 각각에서 나온 것이 **Key**

- 실제로 가져와 섞을 각 token의 정보가 **Value**

Scaled Dot-Product Attention



Multi-Head Attention



- **Scaled Dot-Product Attention**

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

→ Query와 Key의 유사도를 정리한 후 scaling, 이후 softmax를 때려서 가중치로 사용

ex. **it** 은 animal의 정보에 가중치를 높게 뒤서 it + 다른 토큰 정보를 함께 표현함

token	Attention weight
The	0.02
animal	0.63
tired	0.18
because	0.05
run	0.12

- **Multi-Head Attention**

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^o$$

$$\text{where } head_i = Attention(QW_i^Q, KW_i^K, VW_i^K)$$

→ 여러개의 head로 다양한 관계를 학습한 뒤 concat

- **Transformer에서 attention**

Attention	Query	Key / Value	목적
Encoder Self-Attention	Encoder	Encoder	입력 내부 관계

Attention	Query	Key / Value	목적
Decoder Masked Self-Attention	Decoder	Decoder	이전 출력 관계
Encoder-Decoder Attention	Decoder	Encoder	출력 생성 시 입력 참고

4. Why Self-Attention

- Self-Attention vs RNN, CNN

Layer	Complexity	Sequential operations	Maximum path length
Self-Attention	$O(n^2d)$	$O(1)$	$O(1)$
Recurrent	$O(nd^2)$	$O(n)$	$O(n)$
Convolutional	$O(knd^2)$	$O(1)$	$O(\log_k n)$

→ RNN은 sequence 길이에 따라 순차 연산이 필요해 병렬화가 제한되고, CNN은 병렬화는 가능하지만 멀리 떨어진 위치를 연결하기 위해 여러 layer를 거쳐야 한다. 반면 Self-Attention은 한 layer에서 모든 위치를 직접 연결할 수 있어 sequential operation과 maximum path length가 상수 수준이라는 장점을 가진다. 다만 sequence 길이에 대해 $O(n^2)$ 의 계산량을 가지므로 매우 긴 sequence에서는 부담이 될 수 있다.

plus, Attention weight를 통해 일부 관계를 관찰할 수 있어 **해석 가능성**도 기대할 수 있다.

7. Conclusion

- 기존 sequence modeling에서 필수적으로 여겨지던 RNN이나 CNN을 제거하고, Self-Attention을 중심으로 구성한 Transformer architecture를 제안한다.
- 기존 RNN은 이전 hidden state에 의존하기 때문에 순차 계산이 필요해 병렬화가 어렵고, CNN은 멀리 떨어진 위치 사이의 관계를 학습하려면 여러 layer를 거쳐야 한다.
- Transformer는 Self-Attention을 통해 모든 token 간의 관계를 직접 계산하며, Multi-Head Attention으로 다양한 관계를 동시에 학습한다. 또한 순서 정보가 사라지는 문제는 Positional Encoding으로 보완한다. 이러한 구조는 높은 병렬화와 짧은 dependency path를 가능하게 했으며, 실제 기계 번역에서 기존 모델보다 적은 학습 비용으로 SOTA 성능을 달성했다.
- 향후 attention-based model을 text뿐만 아니라 image, audio, video 등 다른 modality로 확장할 가능성도 제시한다.